

SENA

Servicio Nacional de Aprendizaje

TALLER N° 4

POLIMORFISMO

Programación Orientada a Objetos en Java

Pilar de la POO: Polimorfismo

Lenguaje: Java

Nivel: Básico

Ejercicios: 5 ejercicios prácticos + 1 reto integrador

Metodología: Constructivista (aprender-haciendo)

Modalidad: Clases y métodos main en archivos separados

ELABORADO POR

Carlos Arturo Barrientos Suares

*Ingeniero de Sistemas · Especialista en Desarrollo de Software · Máster en Ingeniería de Software y Sistemas
Informáticos*

Marco conceptual del taller

¿Qué es el polimorfismo?

La palabra **polimorfismo** viene del griego y significa "**muchas formas**". Piensa en la palabra "*hablar*": un perro *habla* ladrando, un gato maullando, una persona pronunciando palabras. Una misma acción, muchas formas de realizarla según quién la ejecute.

En POO, el polimorfismo significa que **una misma llamada a un método puede producir comportamientos distintos** según el objeto real que la reciba. Es el pilar que hace que la POO sea realmente poderosa.

¿Cómo se aplica en Java?

En Java existen dos formas principales de polimorfismo:

- **Sobrescritura (Override):** una clase hija redefine un método heredado del padre. Cuando se invoca ese método a través de una referencia del padre, Java ejecuta la versión de la hija real. Se llama **polimorfismo dinámico**.
- **Sobrecarga (Overload):** una misma clase tiene varios métodos con el mismo nombre pero con **diferentes parámetros**. Se llama **polimorfismo estático**.

La forma más potente es asignar un objeto hijo a una variable del tipo padre, por ejemplo: *Animal a = new Perro();* Al invocar *a.hacerSonido()*, Java ejecuta la versión del Perro.

¿Por qué es tan importante?

1. **Código flexible y extensible:** se puede tratar a muchos objetos diferentes con un mismo código.
2. **Menos condicionales:** en lugar de muchos *if/else* según el tipo, se invoca un método y Java decide qué versión ejecutar.
3. **Listas heterogéneas:** se pueden almacenar objetos de distintas clases hijas en una sola lista del tipo padre.
4. **Diseño preparado para el futuro:** agregar nuevas hijas no rompe el código existente.
5. **Base de los patrones de diseño:** el polimorfismo es la base de la programación profesional moderna (Strategy, Factory, Observer, etc.).

IDEA CLAVE

El polimorfismo se apoya en la **herencia** y la **sobrescritura de métodos** que ya viste en los talleres anteriores. En estos ejercicios usaremos referencias del tipo padre apuntando a objetos hijos para apreciar el verdadero poder del polimorfismo. Este es el pilar que **lo une todo**.

Ejercicio 1: Coro de Animales (Sobrescritura)

SITUACIÓN PROBLEMA

Recuperemos las mascotas de la veterinaria del Taller 2. Ahora quieres hacer algo especial: un **coro de animales** donde todos hagan su sonido, uno tras otro. Tienes perros, gatos y vacas. Sin polimorfismo, tendrías que escribir un *if* para cada tipo. Pero con polimorfismo, un mismo bucle sirve para todos.

Tu tarea: crea la clase padre **Animal** y las hijas **Perro**, **Gato** y **Vaca**. Luego almacénalas en un arreglo del tipo `Animal[]` y recórrelo.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. ¿Cómo puede un arreglo del tipo `Animal[]` contener objetos *Perro*, *Gato* y *Vaca*?
2. Cuando el bucle llama a `a.hacerSonido()`, ¿cómo sabe Java qué versión ejecutar?
3. Si mañana agregas **Loro**, ¿tendrás que modificar el bucle principal?

Solución guiada

Archivo: Animal.java

```
public class Animal {
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void hacerSonido() {
        System.out.println(nombre + " hace un sonido generico.");
    }
}
```

Archivo: Perro.java

```
public class Perro extends Animal {
    public Perro(String nombre) { super(nombre); }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + ": Guau guau!");
    }
}
```

Archivo: Gato.java

```
public class Gato extends Animal {
    public Gato(String nombre) { super(nombre); }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + ": Miau!");
    }
}
```

Archivo: Vaca.java

```
public class Vaca extends Animal {
    public Vaca(String nombre) { super(nombre); }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + ": Muuu!");
    }
}
```

Archivo: MainCoro.java

```
public class MainCoro {
    public static void main(String[] args) {
        // POLIMORFISMO: arreglo del tipo padre con objetos hijos
        Animal[] animales = {
            new Perro("Firulais"),
            new Gato("Michi"),
            new Vaca("Lola")
        };

        // Un solo bucle sirve para TODOS los tipos de animal
        for (Animal a : animales) {
            a.hacerSonido(); // Java elige la version correcta
        }
    }
}
```

BENEFICIO APLICADO

El bucle *for* ignora qué tipo concreto es cada animal. Solo llama a *hacerSonido()* y Java escoge la versión correcta en tiempo de ejecución. Esto es **polimorfismo dinámico puro**: un mismo código, muchos comportamientos. Sin esto, tendrías que escribir un *if* por cada tipo de animal (código repetitivo y difícil de mantener).

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una cuarta clase hija **Loro** que diga: "[nombre]: Hola, hola, hola!". Añade dos loros al arreglo sin modificar el bucle. Comprueba que **el bucle funciona automáticamente** con el nuevo tipo. Ese es el poder del polimorfismo.

Ejercicio 2: Calculadora con Sobrecarga

SITUACIÓN PROBLEMA

Estás construyendo una calculadora sencilla. Quieres que el método **sumar** pueda funcionar en varios escenarios:

- Sumar dos números enteros
- Sumar tres números enteros
- Sumar dos números decimales
- Concatenar dos cadenas de texto (también es una "suma")

Sin polimorfismo tendrías que crear métodos con nombres distintos: *sumarDosEnteros*, *sumarTresEnteros*, *sumarDecimales*, *concatenar*. Pero con **sobrecarga** todos se pueden llamar **sumar**.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Piensa:

1. ¿Cómo diferencia Java entre 4 métodos que se llaman igual?
2. ¿Qué es más legible: *calc.sumar(2, 3)* o *calc.sumarDosEnteros(2, 3)*?
3. ¿Puede haber dos métodos con el mismo nombre y los mismos parámetros?

Solución guiada

Fíjate: los 4 métodos se llaman *sumar*, pero **los parámetros son diferentes** (cantidad o tipo). Java usa esa firma para decidir cuál ejecutar en cada llamada.

Archivo: Calculadora.java

```
public class Calculadora {  
  
    // Suma de 2 enteros  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
  
    // Suma de 3 enteros  
    public int sumar(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Suma de 2 decimales  
    public double sumar(double a, double b) {  
        return a + b;  
    }  
  
    // Concatenacion de cadenas  
    public String sumar(String a, String b) {  
        return a + " " + b;  
    }  
}
```

Archivo: MainCalculadora.java

```
public class MainCalculadora {  
    public static void main(String[] args) {  
        Calculadora calc = new Calculadora();  
  
        System.out.println("2 + 3 = " + calc.sumar(2, 3));  
        System.out.println("1 + 2 + 3 = " + calc.sumar(1, 2, 3));  
        System.out.println("2.5 + 3.7 = " + calc.sumar(2.5, 3.7));  
        System.out.println("Hola + Mundo = " + calc.sumar("Hola", "Mundo"));  
    }  
}
```

BENEFICIO APLICADO

El mismo nombre de método *sumar* sirve para 4 operaciones distintas. Java decide cuál ejecutar según los argumentos enviados. Esto hace el código **más legible** y elegante que tener *sumarDosEnteros*, *sumarTresEnteros*, etc. Es el **polimorfismo estático**: se resuelve en tiempo de compilación.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade dos nuevas sobrecargas al método *sumar*:

(a) Sumar un entero con un decimal: *sumar(int, double)* que devuelve *double*.

(b) Sumar un arreglo de enteros: *sumar(int[] numeros)* que devuelve la suma total del arreglo (usa un *for* para recorrerlo).

Prueba las nuevas sobrecargas en el *main*.

Ejercicio 3: Calculadora Polimórfica de Áreas

SITUACIÓN PROBLEMA

Retomamos las figuras geométricas del Taller 3. Ahora quieres calcular el **área total** de un grupo de figuras variadas (cuadrados, círculos, triángulos) sin importar cuántos tipos haya. La solución elegante es guardar todas las figuras en un arreglo del tipo padre y usar polimorfismo.

Tu tarea: crea la clase padre **Figura** y las hijas **Cuadrado**, **Circulo** y **Triangulo**. Almacénalas en *Figura[]* y calcula el total.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. ¿Cómo puede un método sumar áreas **sin saber** qué tipo de figura hay?
2. Si el arreglo tuviera 100 figuras diferentes, ¿el código cambiaría?
3. ¿Qué principio SOLID se aplica aquí (agregar nuevas figuras sin modificar el código existente)?

Solución guiada

Archivo: **Figura.java**

```
public class Figura {  
    public double calcularArea() {  
        return 0;  
    }  
}
```

Archivo: **Cuadrado.java**

```
public class Cuadrado extends Figura {  
    private double lado;  
  
    public Cuadrado(double lado) {  
        this.lado = lado;  
    }  
  
    @Override  
    public double calcularArea() {  
        return lado * lado;  
    }  
}
```

Archivo: Circulo.java

```
public class Circulo extends Figura {
    private double radio;

    public Circulo(double radio) {
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

Archivo: Triangulo.java

```
public class Triangulo extends Figura {
    private double base, altura;

    public Triangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return (base * altura) / 2;
    }
}
```

Archivo: MainAreas.java

```
public class MainAreas {
    public static void main(String[] args) {
        Figura[] figuras = {
            new Cuadrado(4),
            new Circulo(3),
            new Triangulo(5, 6)
        };

        double areaTotal = 0;
        for (Figura f : figuras) {
            double area = f.calcularArea();
            System.out.println("Area parcial: " + area);
            areaTotal += area;
        }

        System.out.println("AREA TOTAL: " + areaTotal);
    }
}
```

BENEFICIO APLICADO

El bucle suma el área total **sin saber** qué figuras hay realmente. Si mañana agregamos *Pentagono*, *Trapecio* o *Hexagono*, este código **no cambia ni una línea**. Eso es **código preparado para el futuro**: abierto a extensión, cerrado a modificación (Principio Open/Closed de SOLID).

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una cuarta clase hija **Rectangulo** con ancho y alto. Área = ancho × alto. Añade dos rectángulos al arreglo **sin tocar el bucle**. Comprueba que el total se calcula correctamente. **Reto extra**: agrega un método **encontrarMayorArea()** que recorra el arreglo y devuelva la figura con mayor área.

Ejercicio 4: Sistema de Nómina Polimórfico

SITUACIÓN PROBLEMA

Una empresa colombiana tiene **empleados de distintos tipos** y quiere calcular el **total de la nómina mensual**. Hay:

- **EmpleadoTiempoCompleto**: con salario mensual fijo.
- **EmpleadoFreelance**: gana por proyectos entregados (proyectos × pago por proyecto).

El objetivo es que Recursos Humanos pueda meter a todos en una lista y obtener el total sin importar qué tipo sean.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. ¿Cómo puede el sistema de nómina calcular el total sin saber si cada empleado es de planta o freelance?
2. Si mañana se agrega el rol **EmpleadoPorHoras**, ¿tienes que modificar el método que calcula el total?
3. ¿Cuál es la ventaja de tener *Empleado[]* en lugar de arreglos separados por tipo?

Solución guiada

Archivo: **Empleado.java**

```
public class Empleado {
    protected String nombre;

    public Empleado(String nombre) {
        this.nombre = nombre;
    }

    public double calcularSalario() {
        return 0;
    }

    public String getNombre() { return nombre; }
}
```

Archivo: EmpleadoTiempoCompleto.java

```
public class EmpleadoTiempoCompleto extends Empleado {
    private double salarioMensual;

    public EmpleadoTiempoCompleto(String nombre, double salario) {
        super(nombre);
        this.salarioMensual = salario;
    }

    @Override
    public double calcularSalario() {
        return salarioMensual;
    }
}
```

Archivo: EmpleadoFreelance.java

```
public class EmpleadoFreelance extends Empleado {
    private int proyectos;
    private double pagoPorProyecto;

    public EmpleadoFreelance(String nombre, int proyectos, double pago) {
        super(nombre);
        this.proyectos = proyectos;
        this.pagoPorProyecto = pago;
    }

    @Override
    public double calcularSalario() {
        return proyectos * pagoPorProyecto;
    }
}
```

Archivo: MainNomina.java

```
public class MainNomina {
    public static void main(String[] args) {
        Empleado[] nomina = {
            new EmpleadoTiempoCompleto("Ana", 2500000),
            new EmpleadoTiempoCompleto("Luis", 3100000),
            new EmpleadoFreelance("Sofia", 4, 800000),
            new EmpleadoFreelance("Carlos", 2, 1200000)
        };

        double totalNomina = 0;
        for (Empleado e : nomina) {
            double pago = e.calcularSalario();
            System.out.println(e.getNombre() + ": $" + pago);
            totalNomina += pago;
        }

        System.out.println("TOTAL NOMINA: $" + totalNomina);
    }
}
```

BENEFICIO APLICADO

En un sistema real, los tipos de empleados pueden cambiar o aumentar. Gracias al polimorfismo, el método que calcula el total de la nómina **nunca necesitará modificarse**: siempre llamará a *calcularSalario()* y cada tipo de empleado responderá a su manera. Este patrón se usa en sistemas reales de RRHH, contabilidad y finanzas.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una tercera clase hija **EmpleadoPorHoras** con **horas trabajadas** y **tarifa por hora**. Su cálculo debe ser $\text{horas} \times \text{tarifa}$. Añade dos empleados por horas al arreglo **sin tocar el bucle**. Verifica que el total se recalcula correctamente.

Ejercicio 5: Sistema de Transporte con instanceof

SITUACIÓN PROBLEMA

Una empresa de transporte en Cúcuta tiene distintos vehículos en su flota. Todos pueden **moverse**, pero cada uno lo hace de forma distinta. Además, algunos vehículos tienen funciones específicas (por ejemplo, solo los buses abren puertas para pasajeros).

La flota incluye:

- **Bus:** transporta muchos pasajeros. Tiene un método especial **abrirPuertas()**.
- **Taxi:** transporta 1 pasajero.
- **Bicicleta:** transporta al conductor.

Cuando recorres la flota, si el vehículo es un Bus, debes abrir sus puertas.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. Si tengo una variable *Vehiculo v* que apunta a un Bus, ¿puedo llamar a *v.abrirPuertas()*?
2. ¿Qué operador de Java me permite **saber el tipo real** del objeto?
3. ¿Cómo hago un "casting" para tratar a la variable como un Bus?

Solución guiada

El operador **instanceof** nos dice si un objeto es realmente de un tipo específico. Cuando lo es, hacemos **casting** para poder usar los métodos propios de ese tipo.

Archivo: Vehiculo.java

```
public class Vehiculo {
    protected String placa;

    public Vehiculo(String placa) {
        this.placa = placa;
    }

    public void mover() {
        System.out.println("El vehiculo " + placa + " se mueve.");
    }

    public String getPlaca() { return placa; }
}
```

Archivo: Bus.java

```
public class Bus extends Vehiculo {
    public Bus(String placa) { super(placa); }

    @Override
    public void mover() {
        System.out.println("Bus " + placa +
            " avanza con 40 pasajeros a 60 km/h.");
    }

    // Metodo propio de Bus (no lo tienen los otros)
    public void abrirPuertas() {
        System.out.println("Bus " + placa + ": puertas abiertas.");
    }
}
```

Archivo: Taxi.java

```
public class Taxi extends Vehiculo {
    public Taxi(String placa) { super(placa); }

    @Override
    public void mover() {
        System.out.println("Taxi " + placa +
            " transporta a 1 pasajero por la ciudad.");
    }
}
```

Archivo: Bicicleta.java

```
public class Bicicleta extends Vehiculo {
    public Bicicleta(String placa) { super(placa); }

    @Override
    public void mover() {
        System.out.println("Bicicleta " + placa +
            " es pedaleada a 20 km/h.");
    }
}
```

Archivo: MainTransporte.java

```
public class MainTransporte {
    public static void main(String[] args) {
        Vehiculo[] flota = {
            new Bus("BUS-001"),
            new Taxi("TAX-555"),
            new Bicicleta("BIC-777"),
            new Bus("BUS-002")
        };

        // Polimorfismo: cada uno responde a su manera
        for (Vehiculo v : flota) {
            v.mover();

            // instanceof + casting para acceder a metodos propios de Bus
            if (v instanceof Bus) {
                Bus b = (Bus) v;
                b.abrirPuertas();
            }
        }
    }
}
```

BENEFICIO APLICADO

Una sola variable de tipo *Vehiculo* puede apuntar a un Bus, Taxi o Bicicleta. El operador *instanceof* permite saber el tipo real cuando necesitamos acceder a métodos específicos (como *abrirPuertas()*). Este patrón es muy frecuente en aplicaciones reales (apps de movilidad como Uber, Rappi, InDriver).

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una cuarta clase **MotoTaxi** con un método propio **recogerPasajero()**. En el main, añade dos motos taxi a la flota y modifica el bucle para que **también** ejecute *recogerPasajero()* cuando el vehículo sea una MotoTaxi. Usa *instanceof*.

Reto Integrador: Sistema de Reservas de Viaje

SITUACIÓN PROBLEMA

Una agencia de viajes en Cúcuta gestiona reservas de diferentes tipos:

- **ReservaVuelo:** nacional o internacional. Precio = distancia × 500 + \$150.000.
- **ReservaHotel:** con noches de estadía. Precio = noches × precioPorNoche.
- **ReservaBusIntermunicipal:** Ruta origen-destino en Colombia. Precio fijo por ruta (consúltalo).

El sistema debe permitir crear un carrito con reservas **de tipos mezclados**, calcular el **total del carrito**, y para las reservas de vuelo, mostrar si es internacional (con un método específico usando `instanceof`).

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Antes de resolver:

1. ¿Qué método debe estar en la clase padre *Reserva*?
2. ¿Qué tipo de arreglo o lista usarás para el carrito?
3. ¿Cómo diferenciarás un vuelo internacional de uno nacional al recorrer el carrito?

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Este reto integrador combina **TODO** lo aprendido:

1. Diseña la clase padre **Reserva.java** con atributos comunes y método *calcularPrecio()*.
2. Diseña las tres hijas: **ReservaVuelo.java** (con atributo *boolean internacional* y *distanciaKm*), **ReservaHotel.java** (con *noches* y *precioPorNoche*), **ReservaBusIntermunicipal.java** (con *origen*, *destino* y *precioFijo*).
3. Cada hija sobrescribe *calcularPrecio()*.
4. Solo *ReservaVuelo* tiene un método propio *esInternacional()*.
5. Diseña un **MainAgencia.java** que:
 - Cree un arreglo mezclado de al menos 5 reservas
 - Recorra el arreglo mostrando cada precio y sumando el total
 - Cuando sea una *ReservaVuelo*, indique si es internacional (usa *instanceof*)
 - Muestre al final el total del carrito

Entrega: los cinco archivos .java funcionando. Este reto pone a prueba tu dominio de los cuatro pilares de la POO.

Cierre metacognitivo del taller

REFLEXIÓN METACOGNITIVA — ¿QUÉ APRENDÍ?

Antes de terminar, tómate 5 minutos para responder en tu cuaderno:

1. Con tus propias palabras, ¿qué es el polimorfismo? Usa una analogía distinta a las del taller.
2. ¿Cuál es la diferencia entre **sobrescritura** (override) y **sobrecarga** (overload)?
3. ¿En qué ejercicio sentiste que el polimorfismo te ahorra **más código**?
4. ¿Para qué sirve el operador *instanceof*? ¿Y el "casting"?
5. Menciona un ejemplo **de tu vida cotidiana** donde una misma "orden" produzca resultados distintos según quien la reciba.
6. Después de estos 4 talleres, ¿cuál de los pilares (encapsulamiento, herencia, abstracción, polimorfismo) sientes que entiendes mejor? ¿Cuál te generó más dudas?
7. ¿Qué duda te quedó? Anótala para consultarla con tu instructor.

Conclusión del ciclo completo de talleres

¡**Felicidades!** Has completado el ciclo de los **cuatro pilares de la POO**. El polimorfismo es la pieza que hace que la POO sea verdaderamente poderosa: gracias a él, un mismo código puede operar sobre **muchas formas distintas** de objetos. Lo viste a través de la **sobrescritura** (clases hijas que redefinen comportamientos), la **sobrecarga** (mismo nombre, diferentes parámetros) y las **referencias del tipo padre** que apuntan a hijos.

Junto al **encapsulamiento**, la **herencia** y la **abstracción**, ahora tienes las bases para construir **software profesional en Java**. El siguiente paso es practicar mucho: cada aplicación real es una combinación creativa de estos cuatro pilares.

*SENA - Servicio Nacional de Aprendizaje
Formación Profesional Integral
Programa: Análisis y Desarrollo de Software*